Physical Design Laboratory Manual: ALU Implementation and Signoff Flow

Department of Electronics and Communication Engineering

1 Introduction

This document details the physical design flow for a 4-bit Arithmetic Logic Unit (ALU) using the SkyWater 130nm PDK. It provides step-by-step instructions for two parallel laboratory environments: Ubuntu (using OpenLane 1.x via Docker) and Mac (using LibreLane / OpenLane 2 via Nix).

2 Step 1: RTL Design and Testbench

Create the following Verilog files inside your `src` directory.

2.1 ALU Design (`alu.v`)

```
`timescale 1ns/1ps

module alu(
  input wire [3:0] a, b,
  input wire [2:0] opcode,
  output reg [3:0] result
);

  always @(*) begin
    case (opcode)
      3'b000: result = a + b;
      3'b001: result = a - b;
      3'b010: result = a & b;
      3'b011: result = a | b;
      3'b100: result = a ^ b;
      3'b101: result = ~a;
      3'b110: result = a<<1;
      3'b111: result = a>>1;
      default: result = 4'd0;
    endcase
  end
endmodule
```

2.2 Testbench (`alu_tb.v`)

```
`timescale 1ns/1ps
module alu_tb();
  reg [3:0] a, b;
  reg [2:0] opcode;
  wire [3:0] result;
```

```
  integer i;
  alu uut(a, b, opcode, result);
  initial begin
    $dumpfile("alu_4bit.vcd");
    $dumpvars(1,alu_tb);
    $monitor("Time = %t | A = %b | B = %b | Op = %b | Res = %b",$time,
        ↪ a, b, opcode, result);

    a = 4'b1100;
    b = 4'b0101;
    for(i=0; i<8; i=i+1) begin
      opcode = i;
      #10;
    end
    $finish;
  end
endmodule
```

3 Step 2: Design Configuration

For this lab, we use a unified configuration file for both Mac and Ubuntu. Because this is a purely combinational ALU without sequential flip-flops, the clock port is set to `null`. Core utilization is set to a low 5% to provide ample routing breathing room and eliminate congestion.

Create `config.json` in your design directory:

```
{
    "DESIGN_NAME": "alu",
    "VERILOG_FILES": ["dir::src/alu.v"],
    "CLOCK_PORT": null,
    "FP_CORE_UTIL": 5,
    "FP_PDN_VPITCH": 25,
    "FP_PDN_HPITCH": 25
}
```

4 Step 3: Flow Execution

4.1 Ubuntu (OpenLane 1.x Interactive Flow)

Launch the Docker container and start the interactive flow:

```
make mount
./flow.tcl -interactive
prep -design alu
```

Run through the physical design stages up to routing and signoff:

```
run_synthesis
run_floorplan
run_placement
run_cts
run_routing
run_parasitics_sta
run_irdrop
run_magic
run_magic_drc
run_magic_spice_export
```

```
run_lvs
save_views
```

Once complete, type `exit` to return to the standard Ubuntu terminal.

4.2 Ubuntu (OpenLane 1.x Automated Flow)

Launch the Docker container and type the following:

```
make mount
./flow.tcl -design alu
```

Error / Troubleshooting: Ubuntu: Failed to compare PDKs / sky130A not found

If you encounter an error stating `sky130A not found` inside the Docker container, it means the PDK is either not installed or not mounted correctly.
Solution: Complete the setup from your host terminal.

1. Exit the OpenLane container: `exit`
2. Install the PDK on your host machine (this downloads and compiles `sky130A`):
   ```
   make pdk
   ```
3. Restart the OpenLane container using `make mount` to ensure the directory is properly mounted, then run the flow again.

Error / Troubleshooting: Ubuntu: Permission Denied during `make pdk`

No, you should not run this as root. If you receive a permission denied error in your home directory, a previous command was likely run with `sudo`, locking you out. Take back ownership by running:

```
sudo chown -R $USER:$USER /home/student/.ciel
```

After running that, try `make pdk` again *without* using `sudo`.

4.3 Mac (LibreLane Automated Flow)

Navigate to the directory containing your configuration file and execute the modern LibreLane toolchain wrapper:

```
librelane config.json
```

Error / Troubleshooting: Mac: PDK sky130A not found

On macOS, LibreLane usually provisions the `sky130A` PDK dependencies automatically through the `nix-shell` environment. If the tool complains that the PDK is missing, it is either because you ran the command outside of the Nix shell, or your Volare PDK manager is installed but has not downloaded the actual PDK data yet.

Solution 1: Activate Nix Shell Ensure you are running inside the correct environment:

```
# Enter the Nix environment
nix-shell

# Run LibreLane again
librelane config.json
```

Solution 2: Manual Volare Setup If the shell is active but the PDK is still missing, set up Volare manually to pull the Sky130 data:

```
# Define your local PDK root path
export PDK_ROOT=~/.volare

# Instruct Volare to download and enable the sky130A PDK
volare enable --pdk sky130A
```

Wait for the libraries to finish extracting, then re-run `librelane config.json`.

5 Step 4: Visualizing the GDS Layout (KLayout)

The GDSII (`.gds`) file is the final manufacturing blueprint of your chip. You will view this physical layout using KLayout.

5.1 Locating the GDS File

Because the exact folder structure can vary slightly depending on the flow version, navigate through your directories to find the generated `alu.gds` file:

- **Ubuntu (OpenLane 1.x):** Go to your latest `RUN_` folder, then into `results`, then `final`, and then `gds`. (Path example: `runs/RUN_DATE/results/final/gds/alu.gds`)
- **Mac (LibreLane):** Go to your latest `RUN_` folder, then into the `final` directory to locate your `alu.gds` file. (Path example: `runs/RUN_DATE/final/alu.gds`)

To open the layout, type the following command in your terminal (adjust the path to match your actual file location):

```
klayout alu.gds
```

5.2 Ubuntu: Loading the Technology Layer Properties File (.lyp)

When KLayout opens for the first time, it runs as a blank template editor with an empty canvas showing tools like ARC, CIRCLE, and DONUT. To display actual colored semiconductor layout structures, we must load the foundry layer parameters.

1. In the KLayout top menu bar, select **File** → **Load Layer Properties**.
2. The navigation file dialog will open. Because the configuration folders are hidden inside your Linux home space, click anywhere inside the file browser selector window and press `Ctrl + H` on your keyboard to reveal hidden files starting with a dot.

3. Direct your path selection index to move through this exact folder directory line:

   `/home/student/.ciel/sky130A/libs.tech/klayout/tech/`

4. Select and open the configuration palette file named **`sky130A.lyp`**.

The right-hand panel indices will instantly fill with real semiconductor labels like `li1` (Local Interconnect), `met1` (Metal 1), and `poly` (Polysilicon), paired with their official foundry color map layouts.

5.3 Mac: Loading the Technology Layer Properties File (.lyp)

For macOS users running LibreLane, the PDK is typically managed by Volare and stored in a hidden directory.

1. In the KLayout top menu bar, select **File** → **Load Layer Properties**.
2. To view hidden folders in the Mac file dialog, press **`Cmd + Shift + .`** (Command + Shift + Period) on your keyboard.
3. Navigate to your Volare PDK installation path:

   `~/.volare/sky130A/libs.tech/klayout/tech/`

4. Select and open the configuration palette file named **`sky130A.lyp`**.

5.4 Viewing Hidden Files and Hierarchy

By default, KLayout only shows the top-level boundary box of the standard cells. To see the hidden internal shapes and transistors:

1. Press **Shift + F** (or go to **Display** → **Full Hierarchy**) to reveal all internal cell details.
2. To clean up the view, open the **Cell Tree** panel on the left side of KLayout. Search for `fill` (filler cells), `decap`, and `tap`. Right-click these cells and select **Hide**. This removes the dummy elements so you can clearly see your actual logic gates.

6 Step 5: Visualisation of Heat Maps (OpenROAD GUI)

6.1 Ubuntu (OpenLane 1.x GUI Launch)

Error / Troubleshooting: Cannot connect to X server

Before launching the GUI from the Ubuntu terminal, you must authorize local connections to the X11 server.

```
xhost +local:
openroad -gui
```

6.2 Mac (OpenLane 2 / LibreLane GUI Launch)

Find the generated database and launch the interactive OpenROAD GUI from nix-shell (within OpenLane folder):

```
openroad -gui
```

7 Step 6: Interpreting Signoff Metrics

For final grading and verification, inspect the master `metrics.csv` text report generated at the end of the flow.

- **Ubuntu Path:** `designs/alu/runs/RUN_DATE/reports/metrics.csv`
- **Mac Path:** `runs/RUN_DATE/metrics.csv`

7.1 Design Integrity and DRC Verification

- `magic_drc_error_count`: Must be exactly **0**. If greater than 0, inspect `reports/signoff/drc.rpt` for spacing or width violations.
- `design_lvs_errors_count`: Must be **0**, verifying that the physical layout matches the logical Verilog netlist.

7.2 Power Consumption Metrics

The total power consumption is broken down into dynamic switching, internal cell behavior, and static leakage:

- `power_switching_total`: Dynamic power consumed charging metal wire capacitances.
- `power_internal_total`: Power consumed internally within standard cells during logic level transitions.
- `power_leakage_total`: Static subthreshold leakage current when gates are idle.
- `power_total`: The combined overall power dissipation of the chip.

Note: Combinational Timing Reminder

Because this ALU does not contain sequential elements and `CLOCK_PORT` is set to `null`, standard sequential setup (`timing_setup_ws`) and hold metrics are omitted. The design operates entirely on combinational path propagation delays.